

Contents

AEP-NEM Engineering System v0.1	2
0. 核心摘要	2
1. 为什么需要 AEP-NEM Engineering System	3
2. 核心本体论：从任务到工程生命体	3
2.1 AEP：工程原子	3
2.2 NEM：节点进化分子	4
2.3 E-Chain：工程链	4
2.4 E-Organism：工程生命体	4
2.5 PoEW：工程工作量证明	4
2.6 Gate：进化闸门	5
3. 总体结构图	5
4. AEP：Atomic Engineering Package	5
4.1 定义	5
4.2 AEP 的核心原则	5
4.3 AEP 最小目录结构	6
4.4 AEP 的边界	6
5. NEM：Node Evolution Molecule	6
5.1 定义	6
5.2 NEM 的核心公式	6
5.3 NEM 成熟度模型	6
5.4 NEM 生命周期	7
5.5 NEM 与 Docker / 模块的关系	7
6. E-Chain：Engineering Chain	7
6.1 定义	7
6.2 主线 E-Chain	7
6.3 支线 E-Chain	7
6.4 E-Chain 的状态	8
7. E-Organism：Engineering Organism	8
7.1 定义	8
7.2 工程生命体的特征	8
7.3 E-Organism 与普通项目的区别	8
8. PoEW：Proof of Engineering Work	9
8.1 定义	9
8.2 PoEW 最小内容	9
8.3 PoEW 与挖矿类比	9
9. Gate：Evolution Gate	9
9.1 定义	9
9.2 Gate 判断类型	9
9.3 Gate 的核心问题	10
9.4 Gate 文件结构建议	10
10. AEP-NEM 体系中的状态机	10
10.1 AEP 状态	10
10.2 NEM 状态	10
10.3 E-Chain 状态	11
10.4 E-Organism 状态	11
11. 标准编号体系	11
11.1 AEP 编号	11
11.2 NEM 编号	11
11.3 E-Chain 编号	11
11.4 PoEW 编号	11
11.5 Gate 编号	12
12. 标准目录结构建议	12
12.1 E-Organism 仓库结构	12
12.2 NEM 目录结构	12

12.3 AEP 目录结构	12
13. 应用场景	13
13.1 软件开发	13
13.2 科研实验	13
13.3 AI 任务外包	13
13.4 数据处理	13
13.5 产品迭代	13
13.6 组织协作	13
14. 与传统方法的区别	13
15. 最小实现路线	14
阶段 1: 文档协议	14
阶段 2: ZIP 工程包	14
阶段 3: Validator	14
阶段 4: Runner	14
阶段 5: Registry	14
阶段 6: E-Organism Dashboard	14
16. GitHub 项目建议	14
17. 未来研究方向	15
18. 核心公理	15
公理一: 工程原子公理	15
公理二: 节点分子公理	15
公理三: 链式生长公理	15
公理四: 生命体公理	15
公理五: 证明公理	15
公理六: 闸门公理	15
19. 与 AI 时代任务外包的关系	16
20. 最终概念锁定	16
21. 下一步建议	16

AEP-NEM Engineering System v0.1

中文名: AEP-NEM 原子化工程体系
 副标题: AI 时代的标准化工程做功、节点进化与项目生命体协议
 版本: v0.1 Draft
 创建日期: 2026-06-02
 适用范围: AI 原生工程、软件项目、科研实验、任务外包、多人协作、长期项目管理

0. 核心摘要

AEP-NEM Engineering System 是一套面向 AI 协作时代的原子化工程体系。
 它试图解决一个新的基础问题: 当 AI、人类、云服务器、代码仓库、实验数据和长期项目共同参与工程建设时, 如何把复杂工程拆分为可封装、可执行、可验证、可复用、可升级的标准化工作单元?
 本体系提出六个核心概念:

层级	缩写	英文命名	中文命名
最小做功单元	AEP	Atomic Engineering Package	工程原子包
节点模块	NEM	Node Evolution Molecule	节点进化分子
主线/支线	E-Chain	Engineering Chain	工程链
项目整体	E-Organism	Engineering Organism	工程生命体
工作量证明	PoEW	Proof of Engineering Work	工程工作量证明
验收闸门	Gate	Evolution Gate	进化闸门

一句话定义：

AEP 是工程原子，NEM 是工程分子，E-Chain 是工程链，E-Organism 是工程生命体；PoEW 证明做功，Gate 决定进化。

更完整地说：

AI 时代的复杂工程，可以被拆解为可封装、可执行、可验证的 AEP 工程原子；多个 AEP 组合成可进化、可升级、可重构的 NEM 节点进化分子；多个 NEM 沿主线或支线连接成 E-Chain 工程链；多条 E-Chain 共同构成持续生长的 E-Organism 工程生命体。每一次做功通过 PoEW 留下证据，并通过 Gate 判断是否进入下一阶段。

1. 为什么需要 AEP-NEM Engineering System

传统项目管理通常以「需求 - 任务 - 执行 - 完成」为基本结构。这个结构适合人类团队，但在 AI 协作时代会暴露出明显问题：

1. 任务描述常常模糊，AI 容易误解边界。
2. AI 可以执行，但缺少标准化交付格式。
3. 长期工程会被大量临时对话、临时命令和临时文件冲散。
4. 外包式任务没有清晰验收标准，容易出现“看起来做了，但无法复现”的问题。
5. 一个工程节点不是一次性完成，而是需要不断升级、修复、重构和版本化。
6. 人类、AI、云服务器之间缺少一个共同读取的工程协议。

AEP-NEM Engineering System 不是普通任务管理方法，而是一套 **AI 原生工程协议**。它把工程推进方式从聊天式委托升级为：

```
生成 AEP 工程原子
↓
投放给 AI / 人 / 云服务器执行
↓
产出 PoEW 工程工作量证明
↓
通过 Gate 验收
↓
推动 NEM 节点进化
↓
连接成 E-Chain 工程链
↓
构成 E-Organism 工程生命体
```

它的目标不是减少人的判断，而是让人的战略意图能够被 AI 稳定执行、被工程系统持续累积、被后续节点复用。

2. 核心本体论：从任务到工程生命体

AEP-NEM 体系的基础判断是：

AI 时代的工程不应该继续被理解为一堆待办事项，而应该被理解为由工程原子、工程分子、工程链和工程生命体组成的可进化结构。

传统项目中的“任务”是临时的、脆弱的、不可复用的。AEP-NEM 体系中的工程单元则具有结构、规则、边界、验收和证明。

2.1 AEP：工程原子

AEP 是最小可封装、可执行、可验证、可迁移、可复用的工程做功单元。它像原子一样，不能无限拆分，否则会失去工程完整性。

一个 AEP 应该包含：

- 任务目标
- 输入材料
- 执行规则
- 环境要求
- 输出路径
- 停止条件
- 验收标准
- 报告模板
- 工作量证明

AEP 不是“一个 Markdown 文件”，而是一个工程协议包。初期可以用 zip 封装，内部采用 Markdown、YAML、JSON、PDF 等现有格式实现。

2.2 NEM：节点进化分子

NEM 是由多个 AEP 组合而成的节点进化分子。它不是一次性任务，而是可持续升级的工程模块。

一个 NEM 可以先通过 3 到 5 个 AEP 达到基础可用状态，再通过 10 到 15 个 AEP 达到稳定状态，最后通过 20 个以上 AEP 进入成熟、重构或主线采纳状态。

NEM 具有模块属性：

- 可版本化
- 可升级
- 可重构
- 可依赖
- 可替换
- 可迁移
- 可被其他 NEM 调用

2.3 E-Chain：工程链

E-Chain 是由多个 NEM 按依赖关系连接形成的主线或支线。

主线 E-Chain 推动项目核心能力成长。支线 E-Chain 用于实验、探索、性能分析、边缘技术试探和风险验证。

2.4 E-Organism：工程生命体

E-Organism 是由多条 E-Chain、多组 NEM 和大量 AEP 共同构成的长期生长系统。

一个复杂项目不再只是代码仓库，而是一个不断吸收工程原子、进化工程分子、延展工程链的生命体。

2.5 PoEW：工程工作量证明

PoEW 是每次工程做功后的证据集合。它证明执行者不仅“说完成了”，而且留下了可检查、可复现、可追踪的工程痕迹。

PoEW 可以包括：

- 运行日志
- 修改文件列表
- 输出文件
- 测试结果
- 错误分类
- final_report.md
- validation_report.md
- result_summary.json

2.6 Gate：进化闸门

Gate 是工程体系中的验收机制。它决定一个 AEP 是否通过，一个 NEM 是否升级，一条 E-Chain 是否允许继续向前。

Gate 不只判断成功，也判断：

- PASS：通过
 - HOLD：暂缓，条件不足
 - FAIL：失败，需要修复
 - PARTIAL：部分通过
 - ADOPT：采纳为主线基础
 - REFACTOR：需要重构
-

3. 总体结构图

E-Organism 工程生命体

E-Chain 主线工程链

NEM-001 节点进化分子

AEP-001 工程原子包

AEP-002 工程原子包

AEP-003 工程原子包

Gate-001 进化闸门

NEM-002 节点进化分子

AEP-004 工程原子包

AEP-005 工程原子包

Gate-002 进化闸门

NEM-003 节点进化分子

E-Chain 支线工程链

NEM-S001 实验节点分子

NEM-S002 性能节点分子

NEM-S003 风险验证节点分子

这张图说明：

- AEP 是最小工程原子。
 - NEM 是多个 AEP 构成的工程分子。
 - E-Chain 是多个 NEM 组成的主线或支线。
 - E-Organism 是多条工程链组成的项目生命体。
 - Gate 控制进化方向。
 - PoEW 为每次做功留下证明。
-

4. AEP：Atomic Engineering Package

4.1 定义

AEP（Atomic Engineering Package）是 AI 原生工程体系中的最小标准化做功单元。

它将一个工程任务封装成可迁移、可读取、可执行、可验证的结构化工作包。

4.2 AEP 的核心原则

一个合格的 AEP 必须满足六个原则：

Self-contained	自包含
Portable	可迁移
Agent-readable	AI 可读
Executable	可执行
Verifiable	可验证
Resumable	可恢复

4.3 AEP 最小目录结构

```
AEP_PACKAGE/
  OO_START_HERE.md
  manifest.json
  human_brief.md
  human_brief.pdf
  agent_spec.md
  rules.md
  task_map.yaml
  tasks/
  validation/
  outputs/
  reports/
  changelog.md
```

4.4 AEP 的边界

AEP 不是越大越好。一个 AEP 只应该解决一个明确工程目标。

如果一个 AEP 同时包含写代码、跑长任务、改公式、做 UI、写报告、商业分析，它就已经过大，需要拆分。

AEP 的理想颗粒度是：

一个 AI 或一个工程执行者能够在明确上下文内完成，并能产生明确 PoEW 的最小工程包。

5. NEM: Node Evolution Molecule

5.1 定义

NEM (Node Evolution Molecule) 是由多个 AEP 工程原子组成的可进化工程模块。

NEM 不是静态节点，而是一个可以持续吸收 AEP、不断升级、重构、版本化和被下游依赖的工程分子。

5.2 NEM 的核心公式

$$NEM = \sum(AEP_i \times PoEW_i) + Gate\ Decision + Version\ State$$

其中：

- AEP_i: 第 i 个工程原子包
- PoEW_i: 第 i 个工程包执行后的工作量证明
- Gate Decision: 闸门验收判断
- Version State: 节点分子的当前版本状态

5.3 NEM 成熟度模型

阶段	典型 AEP 数量	状态	含义
v0.1	3-5	BASIC	基础可用

阶段	典型 AEP 数量	状态	含义
v0.3	6-10	STABLE	初步稳定
v0.5	11-15	EXTENDED	能力增强
v1.0	16-25	ADOPTED	主线采纳
v2.0	25+	REFACTORED	重构升级

5.4 NEM 生命周期

```

DRAFT
↓
BASIC
↓
STABLE
↓
EXTENDED
↓
ADOPTED
↓
UPGRADED / REFACTORED / DEPRECATED

```

5.5 NEM 与 Docker / 模块的关系

NEM 类似 Docker 镜像或软件模块，但它不是运行环境，而是工程能力模块。

NEM 可以被依赖、升级、重构、替换。一个项目中的多个 NEM 共同形成工程能力网络。

6. E-Chain: Engineering Chain

6.1 定义

E-Chain 是由多个 NEM 连接形成的工程链。它表示一条主线、支线、实验线或产品线。

6.2 主线 E-Chain

主线 E-Chain 是项目的战略推进链。它通常包含核心工程能力。

例如一个 AI 音乐后处理项目可能包含：

```

Runtime NEM
↓
Benchmark NEM
↓
Sample Library NEM
↓
Processing Preset NEM
↓
Report System NEM
↓
MVP Product NEM

```

6.3 支线 E-Chain

支线 E-Chain 用于探索和验证。它可以反哺主线，但不一定直接进入主线。

支线包括：

- 性能分析链
- 参数实验链
- 边缘技术探索链
- 风险验证链
- 商业假设验证链

6.4 E-Chain 的状态

状态	含义
PLANNED	已规划
ACTIVE	正在推进
HOLD	暂缓
MERGED	已并入主线
ARCHIVED	已归档
ABANDONED	已放弃

7. E-Organism: Engineering Organism

7.1 定义

E-Organism 是由多条 E-Chain、多组 NEM 和大量 AEP 组成的长期工程生命体。它代表一个项目的完整工程生态。

7.2 工程生命体的特征

一个 E-Organism 具有：

- 代谢：不断吸收 AEP 做功
- 生长：NEM 版本不断升级
- 分化：主线和支线分离
- 记忆：PoEW 和报告沉淀
- 免疫：Gate 防止错误进入主线
- 重构：旧 NEM 被替换或升级
- 生态：不同模块之间形成依赖网络

7.3 E-Organism 与普通项目的区别

普通项目通常以“功能完成”为中心。E-Organism 以“持续进化”为中心。

普通项目问：

这个功能做完了吗？

E-Organism 问：

这个 NEM 进化到哪个版本？

它产生了哪些 PoEW？

它是否通过 Gate？

它是否可以被下游依赖？

8. PoEW: Proof of Engineering Work

8.1 定义

PoEW 是工程工作量证明，用来证明一个 AEP 或 NEM 的执行不是口头完成，而是留下了可验证结果。

8.2 PoEW 最小内容

一个 PoEW 至少应该包含：

执行摘要
执行者
执行时间
输入材料
执行命令
修改文件列表
输出文件列表
日志路径
测试结果
失败分类
验收结论
下一步建议

8.3 PoEW 与挖矿类比

AEP-NEM 体系中的工程做功类似挖矿：

挖矿系统	AEP-NEM 工程体系
区块输入	AEP 输入材料
矿工	AI / 人 / 服务器
算力做功	工程执行
难度目标	validation rules
哈希结果	result_summary.json
出块成功	Gate PASS / ADOPT
出块失败	HOLD / FAIL

PoEW 的意义在于：它让 AI 的工作不只是“回答”，而是变成可验证工程输出。

9. Gate: Evolution Gate

9.1 定义

Gate 是进化闸门，用于判断 AEP、NEM 或 E-Chain 是否可以进入下一阶段。

Gate 不是形式检查，而是防止错误、幻觉、模糊结果和不可复现结果进入主线的机制。

9.2 Gate 判断类型

判断	含义
PASS	当前任务通过
HOLD	暂缓，需要补条件
FAIL	失败，需要修复
PARTIAL	部分通过

判断	含义
ADOPT	采纳为主线基础
REFACTOR	需要重构
DEPRECATE	废弃旧版本

9.3 Gate 的核心问题

Gate 必须回答：

是否完成？
 是否可复现？
 是否有日志？
 是否有报告？
 是否有风险？
 是否可进入下游？
 是否需要重构？

9.4 Gate 文件结构建议

```
GATE_PACKAGE/
  gate_checklist.md
  validation_rules.yaml
  required_outputs.md
  risk_review.md
  adopt_decision.md
  gate_report.md
```

10. AEP-NEM 体系中的状态机

10.1 AEP 状态

状态	含义
DRAFT	草案
READY	可执行
ACTIVE	执行中
PASS	通过
HOLD	暂缓
FAIL	失败
ARCHIVED	归档

10.2 NEM 状态

状态	含义
DRAFT	设计中
BASIC	基础可用
STABLE	稳定可用
EXTENDED	增强扩展
ADOPTED	主线采纳
REFACTORED	重构完成
DEPRECATED	旧版废弃

10.3 E-Chain 状态

状态	含义
PLANNED	已规划
ACTIVE	正在推进
HOLD	暂缓
MERGED	已并入主线
ARCHIVED	归档

10.4 E-Organism 状态

状态	含义
SEED	种子期
GERMINATION	萌芽期
GROWTH	生长期
STABILIZATION	稳定期
SCALE	扩张期
REGENERATION	再生/重构期

11. 标准编号体系

11.1 AEP 编号

AEP-0001
AEP-0002
AEP-0003

在项目内部可加前缀：

MT-001-AEP-001
ST-002-AEP-004

11.2 NEM 编号

NEM-001 Runtime System
NEM-002 Benchmark System
NEM-003 Sample Library

11.3 E-Chain 编号

ECHAIN-MT-001 Engineering Foundation Chain
ECHAIN-ST-001 Performance Experiment Chain

11.4 PoEW 编号

PoEW-AEP-001-20260602
PoEW-NEM-001-v0.3

11.5 Gate 编号

GATE-AEP-001
GATE-NEM-001-v0.1
GATE-ECHAIN-MT-001

12. 标准目录结构建议

12.1 E-Organism 仓库结构

```
engineering-organism/  
  README.md  
  organism_manifest.yaml  
  chains/  
    main/  
    side/  
  nems/  
    NEM-001_Runtime/  
    NEM-002_Benchmark/  
    NEM-003_Sample_Library/  
  aeps/  
    AEP-0001/  
    AEP-0002/  
    AEP-0003/  
  gates/  
  poew/  
  reports/  
  schemas/  
  docs/
```

12.2 NEM 目录结构

```
NEM-001_Runtime/  
  nem_manifest.yaml  
  README.md  
  versions/  
    v0.1_BASIC/  
    v0.3_STABLE/  
    v1.0_ADOPTED/  
  aeps/  
  gates/  
  poew/  
  reports/  
  changelog.md
```

12.3 AEP 目录结构

```
AEP-0001_Runtime_Baseline_Test/  
  00_START_HERE.md  
  manifest.json  
  agent_spec.md  
  rules.md  
  task_map.yaml  
  tasks/  
  validation/
```

outputs/
reports/
changelog.md

13. 应用场景

AEP-NEM Engineering System 可以用于：

13.1 软件开发

把复杂软件拆成多个 NEM，再用 AEP 投放给 AI 或工程师执行。

13.2 科研实验

把一个研究方向拆成实验 NEM，每个实验用 AEP 封装输入、变量、运行方式、结果和报告。

13.3 AI 任务外包

把任务从聊天请求变成工程包，降低沟通误差。

13.4 数据处理

把数据清洗、特征提取、模型评测、报告生成拆成标准 AEP。

13.5 产品迭代

把一个产品能力当成 NEM，持续用 AEP 升级。

13.6 组织协作

多个成员或多个 AI 可以领取不同 AEP，最后通过 Gate 汇入 NEM。

14. 与传统方法的区别

传统方法	AEP-NEM Engineering System
任务清单	工程原子
项目节点	节点进化分子
项目计划	工程链
项目整体	工程生命体
完成汇报	PoEW 工程证明
人工验收	Gate 进化闸门
一次性完成	持续进化
聊天式委托	协议化投放

核心区别是：

AEP-NEM 体系不把工程理解为“做完任务”，而把工程理解为“持续吸收原子、进化分子、延展链条、形成生命体”。

15. 最小实现路线

AEP-NEM 体系不需要一开始开发复杂软件。最小阻力路线是：

阶段 1：文档协议

先定义术语、结构、目录、状态机和验收规则。

阶段 2：ZIP 工程包

用 zip 封装 AEP，用文件夹组织 NEM。

阶段 3：Validator

用 Python 写基础校验器：

检查 manifest 是否存在

检查 task_map 是否存在

检查 validation_rules 是否存在

检查 outputs 和 reports 是否存在

阶段 4：Runner

进一步读取 task_map，执行命令，收集日志，生成 PoEW。

阶段 5：Registry

建立 AEP / NEM / E-Chain 注册表，跟踪版本、依赖和状态。

阶段 6：E-Organism Dashboard

为大型项目生成工程生命体视图，包括节点成熟度、链条状态、风险和下一步 AEP。

16. GitHub 项目建议

可以建立一个独立项目：

aep-nem-engineering-system/

建议结构：

```
aep-nem-engineering-system/  
  README.md  
  docs/  
    AEP_NEM_Engineering_System_v0.1.md  
    AEP_Standard_v0.1.md  
    NEM_Node_Evolution_Molecule_v0.1.md  
    glossary.md  
  schemas/  
    aep_manifest.schema.json  
    nem_manifest.schema.json  
    gate.schema.json  
  examples/  
    moodify_runtime_nem/  
    sample_aep_package/  
  src/
```

```
aep_nem/  
tests/  
LICENSE  
pyproject.toml
```

这个项目可以成为 AEP 和 NEM 的共同母体项目。

17. 未来研究方向

AEP-NEM 体系后续可以继续研究：

1. AEP 的最佳任务颗粒度。
 2. NEM 的成熟度评分模型。
 3. E-Chain 的依赖图与风险传播。
 4. PoEW 的可信度评估。
 5. Gate 的自动化验收规则。
 6. AI 执行者之间的任务交接协议。
 7. 多 AI 协作中的冲突合并机制。
 8. 工程生命体的健康度指标。
 9. AEP Registry 与 NEM Registry。
 10. AEP-NEM 与 GitHub Actions、Docker、CI/CD 的结合。
 11. AEP-NEM 在科研实验中的复现标准。
 12. AEP-NEM 在商业外包中的合同化表达。
-

18. 核心公理

AEP-NEM Engineering System 可以提炼出以下公理：

公理一：工程原子公理

任何复杂工程都可以被拆分为一组可执行、可验证的最小工程原子。

公理二：节点分子公理

单个工程原子无法形成稳定能力，多个工程原子通过结构化组合形成节点进化分子。

公理三：链式生长公理

多个节点进化分子沿依赖关系排列，形成主线或支线工程链。

公理四：生命体公理

长期项目不是静态文件集合，而是由工程原子、工程分子和工程链共同构成的工程生命体。

公理五：证明公理

没有 PoEW 的工程做功不应进入主线，因为它不可验证、不可复现、不可追踪。

公理六：闸门公理

没有 Gate 的进化不是工程进化，而是无约束堆叠。

19. 与 AI 时代任务外包的关系

AEP-NEM 体系可以成为 AI 时代任务外包的基础协议。

传统外包依赖口头沟通、需求文档、会议和人工验收。AI 时代需要一种更结构化的方式：

任务被封装为 AEP

AEP 被投放给 AI / 人 / 云服务器

执行结果生成 PoEW

Gate 进行验收

通过的结果汇入 NEM

NEM 推动 E-Chain 前进

因此，AEP-NEM 体系不仅适用于个人项目，也适用于：

- AI 外包平台
 - 开源协作
 - 企业内部 AI 工程管理
 - 科研任务复现
 - 自动化软件开发
 - 多智能体协作
-

20. 最终概念锁定

AEP-NEM Engineering System 的核心概念正式锁定为：

AEP = Atomic Engineering Package = 工程原子包

NEM = Node Evolution Molecule = 节点进化分子

E-Chain = Engineering Chain = 工程链

E-Organism = Engineering Organism = 工程生命体

PoEW = Proof of Engineering Work = 工程工作量证明

Gate = Evolution Gate = 进化闸门

核心表达：

AEP 是工程原子，NEM 是工程分子，E-Chain 是工程链，E-Organism 是工程生命体。PoEW 证明工程做功，Gate 决定工程进化。

体系定义：

AEP-NEM Engineering System 是一套面向 AI 协作时代的原子化工程体系。它将复杂工程拆解为可封装、可执行、可验证的工程原子，将工程原子组合为可进化的节点分子，再将节点分子连接成工程链，最终形成可持续生长的工程生命体。

最终判断：

在 AI 时代，真正稀缺的不是单次任务执行能力，而是把任务封装为原子、把原子组合为分子、把分子连接为链条、把链条养成生命体的工程组织能力。

21. 下一步建议

下一步应该把本文件作为 AEP-NEM Engineering System v0.1 的体系母本，然后继续推进三件事：

1. 把 AEP Standard v0.1 升级为 v0.2，加入 NEM / E-Chain / PoEW / Gate 概念。
2. 把 NEM Research 项目升级为 AEP-NEM 总项目。
3. 用一个真实项目，例如 Moodify Runtime，制作第一个完整样例：

E-Organism: Moodify

E-Chain: Engineering Foundation Chain

NEM: Runtime Cloud System

AEP: Runtime Baseline / Full Test / State Machine / Gate
PoEW: Test logs + reports + validation summary
Gate: Runtime v0.1 BASIC acceptance

这个样例将成为 AEP-NEM Engineering System 的第一个真实证明。